The Metaflow Lightning Chipset*

Bruce D. Lightner
Metaflow Technologies, Inc.
San Diego, California

Gene Hill
LSI Logic Corporation
Milpitas, California

Abstract: The Lightning SPARC superscalar microprocessor chipset is the first processor to implement the Metaflow architecture. This architecture exploits instruction-level parallelism present in conventional sequential programs by hardware means, without relying on sophisticated optimizing compilers. The Lightning processor is capable of executing instructions out-of-order, and speculatively. Lightning is based on an integrated instruction-shelving structure called the DRIS (Deferred-scheduling, Register-renaming Instruction Shelf). All instructions are placed in the DRIS as they are fetched. A dataflow-based instruction scheduler selects instructions for execution from the DRIS after all needed operands have been computed. Instruction results are shelved in the DRIS along with the instructions. Instructions are removed from the DRIS, and their results written to permanent storage, in the order in which they were fetched. Instructions following conditional branches are executed speculatively, based on a predicted direction; the effects of a mispredicted branch are quickly repaired. Precise traps and interrupts are supported by the architecture, effectively hiding the complexity of out-of-order, speculative execution from the programmer.

1 Introduction

The Lightning superscalar SPARC microprocessor is based on the Metaflow architecture developed by Metaflow Technologies.[1] The Metaflow architecture represents a unified approach to maximizing the performance of conventional sequential instruction sets by employing out-of-order and speculative execution of instructions. The Metaflow architecture also facilitates the construction of processors which can effectively utilize superscalar instruction issue and execution techniques.

A *superscalar* microprocessor is characterized by its ability to execute, on average, more than one instruction per clock period while maintaining binary compatibility with industry standard microprocessors. This capability, of course, implies multiple execution units, and logic to issue more than one instruction per clock period. An alternate approach that yields roughly equivalent performance, and binary compatibility, is to issue and execute a single instruction at a time, but at a much higher clock rate, with more processor pipeline stages—a technique called *superpipelining*. However, both superscalar and superpipelined processors share an inherent limitation: whenever the processor encounters an instruction which depends on the result of an unfinished instruction, it must *stall execution*. The stall introduces clock periods when the processor's execution units are idle, resulting in less than peak performance.

If a program has sufficient instruction-level concurrency ("parallelism"), stalls can be avoided by finding other (independent) instructions to execute while the stall-inducing condition is resolved (i.e., defer dependent instructions until their needed operands are available). Until recently, microprocessor architects have been forced to rely primarily on compilers to exploit instruction level parallelism to avoid stalls. For example, code scheduling and software pipelining are compiler techniques used to *exploit* instruction level parallelism to avoid stalls. Loop unrolling and trace scheduling are sophisticated compiler techniques used to *expose* instruction level parallelism to accomplish the same thing.

Although compiler techniques to expose and exploit instruction level parallelism have proven effective, these techniques are limited by the fact that at

*Development of the Lightning superscalar implementation of the Metaflow out-of-order speculative execution architecture was supported jointly by LSI Logic Corporation and Hyundai Electronics Industries Co., Ltd.

[1]Lightning is a trademark of LSI Logic Corporation, SPARC is a trademark of Sun Microsystems, and Metaflow is a trademark of Metaflow Technologies, Inc.

compile time the execution behavior of the program seldom can be known in detail. Lacking dynamic information regarding a program's data-dependent behavior, compilers must make imperfect judgments when performing code optimization—sometimes even reducing a program's performance instead of increasing it. For example, loop unrolling can actually "pessimize" a program's code in certain circumstances: a small loop iteration count can nullify the benefits of unrolling, an unrolled loop may not fit in the processor's instruction cache, the compiler's register allocation scheme can be overwhelmed, forcing operands to/from memory unnecessarily.

Total reliance on compilers for optimum performance has other drawbacks as well. It requires access to the program's source code. It increases compilation time. It will require subsequent program re-compilations as new processor generations are released. In a network environment, with a mixture of (binary compatible) processor generations, one may be forced to manage multiple copies of the same executable program—each tuned for a particular processor version.

An alternative approach is to shift some of the burden of code scheduling, register allocation, and loop unrolling from the compiler to the hardware, making optimal use of accurate dynamic information regarding the program's behavior. This dynamic information is readily available to the processor at run time. Processors can be constructed to look past dependent instructions, searching for later independent instructions, executing them *out-of-order* without stalling. However, to be effective, out-of-order execution must be coupled with other mechanisms, to solve problems related to imprecise interrupts and traps, artificial dependencies caused by register-set limitations, and stalls caused by data-dependent transfers of control (e.g., conditional branches).

2 Metaflow Architecture

The mechanisms which implement the Metaflow architecture are contained in a unified logic structure known as the DRIS (Deferred-scheduling, Register-renaming Instruction Shelf)[1]. A particular superscalar implementation of the DRIS, optimized for the SPARC instruction set, is shown in Figure 1. The Figure shown is a somewhat simplified diagram of actual logic structures used in the Lightning microprocessor chipset. A hardware structure called the DCAF (Dataflow Content-Addressable FIFO) was developed for the Lightning project to efficiently implement the DRIS hardware mechanisms needed for the Metaflow architecture. The separate branch unit contains other parts of the DRIS structure.

2.1 Out-of-Order Execution

SPARC instructions are predecoded before they are placed in the instruction cache—branch target addresses are pre-calculated and stored in the cache to speed branch instruction execution. The program counter (PC) is used to fetch up to four instructions from the instruction cache during each clock period. Branch instructions are executed by the branch unit directly (one per clock). Non-branch instructions are placed into the DCAF at the rate of up to three per clock. The placing of an instruction in the DCAF is equivalent to instruction issue in a conventional processor.

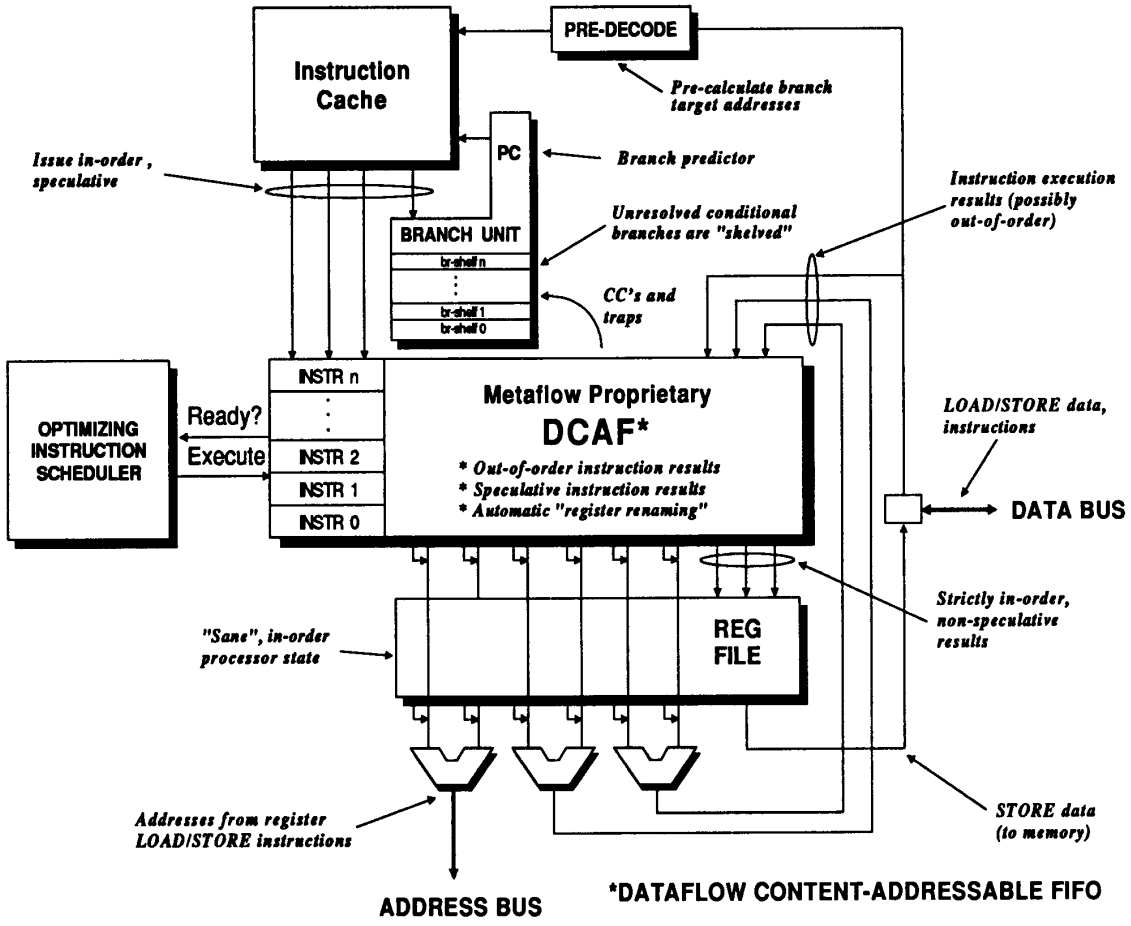
As they are *issued* into the DCAF, instructions' operand dependencies are recorded in the DCAF along with the instruction. For bookkeeping purposes, all instructions are marked with a unique identifier (ID). Instructions are "scheduled" for execution from the DCAF when ready (i.e., when all the instruction's operands are available, and the required execution unit is available). The instruction scheduling is dataflow based. Note that instruction fetch and issue is decoupled from instruction scheduling and execution.

In the example shown in Figure 1, up to three instructions can be placed in execution each clock period. Instructions scheduled for execution receive their operands from either the DCAF or the register file. Instruction execution results are *always* written back into the DCAF. Associated with each instruction in the DCAF is a place to store that instruction's results. While an instruction is held in the DCAF, its results remain in the DCAF.

When it is "safe" to do so, instructions are *retired* by writing their results into the register file or memory. The results of up to three instructions can be written into the register file during each clock. Instructions (and their results) are always retired in strict program order, therefore the register file always contains a "sane", programmer-visible state. In contrast, the DCAF holds a more chaotic program state, due to the out-of-order execution of instructions.

The DCAF is a FIFO (first-in first-out) structure in the sense that instructions are entered (issued) and removed (retired) in the same (program-defined) order. The DCAF is content addressable for the purposes of control and dataflow scheduling.

Figure 1: Dataflow Content-Addressable FIFO (DCAF)



2.2 Register Renaming

Because each instruction in the DCAF has a field to store its results, each time a register is "written" by an instruction, a unique instantiation of that register is created. Subsequent dependent instructions which "read" the value written by the first instruction will use the result field to source the needed operand—provided the first instruction has not yet been retired and its result field written into the register file.

This hardware mechanism results in automatic *register renaming*. Artificial dependencies caused by inadvertent register re-use are automatically eliminated by the DCAF hardware at run time.

2.3 Memory Access

The Lightning microprocessor has a single integer ALU dedicated to memory address calculation. It is used for calculating the effective memory address of register LOAD and STORE instructions. The Metaflow architecture supports out-of-order LOAD requests from memory. STORE requests are *always* sent to memory in strict program-defined order, at the time the STORE instructions are retired from the DCAF. This insures that the processor's memory system always contains the correct ("sane") in-order state.

A LOAD request can be safely issued out-of-order, providing the effective address of the LOAD does not

conflict with preceding incomplete STORE requests. The DCAF is used detect when it is "safe" to issue LOAD requests ahead of STORE requests.

2.4 Precise Traps

In order to provide the required binary compatibility with the SPARC instruction set, the Lightning processor has to respond to interrupts and traps (e.g., page faults, numeric exceptions, etc.) in the proper way. The fact that the processor executes instructions out-of-order must be "hidden" from the programmer. This is accomplished by deferring the processing of traps until the instruction which generated the trap condition is ready to be retired from the DCAF. This insures that all preceding instructions have been completed. At that point, the DCAF signals the branch unit that an exception condition has been detected, and the branch unit begins exception processing. Part of the exception processing is to signal the DCAF to delete all instructions held in the DCAF which logically follow the instruction which caused the trap. This *state repair* operation happens instantly, leaving the DCAF in the proper state to begin accepting instructions along the new execution path (i.e., the trap processing subroutine).

2.5 Speculative Execution

The branch unit shown in Figure 1 provides a *speculative execution* mechanism which allows the Lightning processor to continue fetching and issuing instructions past multiple unresolved conditional branch instructions. In the case where a conditional branch instruction is dependent on an instruction which has not yet been executed by the DCAF hardware (e.g., the condition codes are unknown), the branch unit uses branch history information stored in the instruction cache to predict the direction of the unresolved branch. Unresolved branches are "shelved" in the branch unit until the needed condition codes are known.

If the branch prediction was correct, the branch instruction is removed from the branch unit. If the branch prediction was incorrect, the state of the DCAF needs adjustment because it contains instruction which were fetched, issued, and possibly executed, along an incorrect program path. In this case, the branch unit makes use of the "instant" state repair feature of the DCAF described previously to purge the incorrectly fetched instructions from the DCAF. In parallel with the DCAF state repair operation, the branch unit begins fetching instructions along the known correct program path.

The DCAF never retires instructions which were issued after unresolved conditional branches—all such instructions are *speculative*, and therefore are subject to being flushed from the DCAF as part of a state repair operation. This insures that the register file (and memory) are never affected by speculatively executed instructions until the branch unit has verified that those instructions indeed should have been executed.

3 The Lightning Module

The Lightning SPARC microprocessor chipset is the first implementation of the Metaflow architecture. Lightning is a superscalar processor capable of issuing up to four instructions per clock. Lightning conforms to the SPARC RISC instruction set as defined by Sun Microsystems[2]. The Lightning chipset is packaged as a multi-chip module which contains the four ASIC chips which make up the Lightning processor, plus cache RAM. The module is designed to plug into an 85-pin Mbus Level 2 connector. Figure 2 shows a simplified block diagram of the Lightning module.

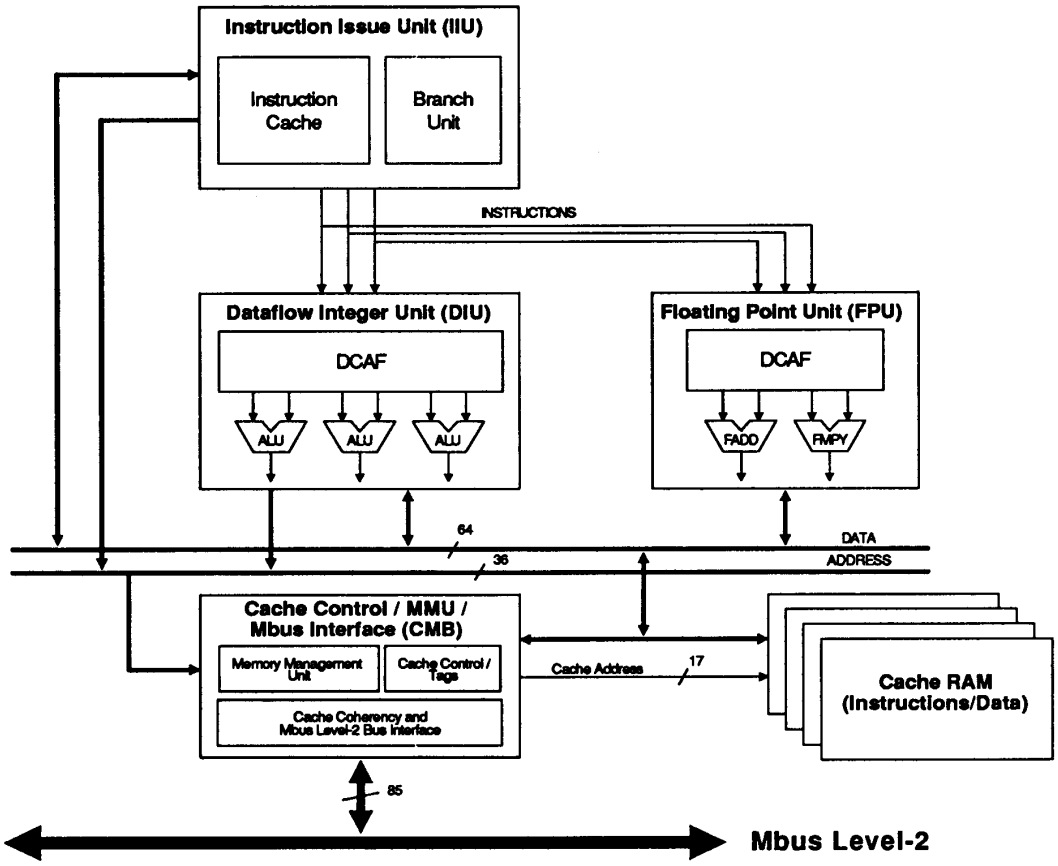
The Lightning module contains an instruction cache, a branch unit, two dynamic register files (the DCAF structure described above), two integer ALUs, a memory address ALU, two floating point ALUs, a memory management unit (MMU), up to one megabyte of external cache memory, and an Mbus Level 2 bus interface. The four Lightning ASICs are the Instruction Issue Unit (IIU), the Dataflow Integer Unit (DIU), the Floating Point Unit (FPU), and the Cache controller/MMU/Bus interface (CMB). The external cache is a first-level cache for data and a second-level cache for instructions.

3.1 Instruction Issue Unit (IIU)

The IIU is a single chip responsible for dispatching up to four instructions each clock cycle. These instructions (any combination of three integer/floating and one branch) are fetched from its on-board instruction cache. Each clock, up to three instructions are sent to the DIU and/or FPU chips, plus one branch instruction can be executed by the IIU internally. The IIU chip is also responsible for branch prediction, speculative execution, and trap processing.

If the IIU chip detects an internal instruction cache miss, it will request data from the second level cache via the CMB chip. The IIU contains a translation look-aside buffer (TLB) which it uses to translate virtual instruction addresses to physical addresses before requesting instructions from the external data/instruction cache.

Figure 2: Lightning SPARC Mbus Module



3.2 Dataflow Integer Unit (DIU)

The DIU chip utilizes a DCAF and multiple functional units to execute up to three integer instructions each clock cycle. Load and store operations are scheduled by the DIU to the module's data cache via the CMB. The DIU contains a TLB which is used to translate virtual addresses to physical addresses before scheduling memory requests via the CMB chip. The DIU chip also contains circuitry which retires instructions in-order and provides for processor state repair (used to implement precise interrupts and traps, and branch repair).

3.3 Floating Point Unit (FPU)

The FPU chip consists of a DCAF and two pipelined floating point functional units: an adder/subtracter and a multiplier. The FPU can start one floating point operation per clock cycle using either of the functional units. In the same clock the FPU chip can execute one floating point register load/store operation. Floating point register load/store operations are initiated by the DIU chip, and completed by the CMB and FPU chips.

3.4 Cache Control/MMU/Bus Interface (CMB)

The CMB chip provides an Mbus Level 2 interface to the outside world, maintains the module's data/instruction cache, and services TLB misses for the IIU and DIU. The CMB chip contains the Lightning module's cache tag RAM. The CMB services IIU instruction requests, DIU/FPU data requests, and

Mbus cache coherency requests.

The CMB chip implements a "write invalidate" cache coherency protocol between the cache and the Mbus. The CMB also contains a SPARC Reference MMU compatible controller which updates the IIU and the DIU TLBs by accessing operating system page tables.

3.5 External Cache

The module's data/instruction cache consists of discrete RAM chips. It is a physically addressed, direct-mapped cache accessed by the IIU, DIU/FPU, and Mbus via the CMB. This cache serves as a primary data cache and secondary instruction cache. It is 64 bits wide and may contain up to 1 Mbyte of data, depending upon the RAM technology utilized.

4 Conclusions

The Metaflow architecture effectively exploits the instruction-level parallelism available in conventional sequential programs using hardware mechanisms—without relying on sophisticated optimizing compilers. The hardware sets aside instructions that would otherwise cause stalling due to data or control dependencies, proceeding immediately to issue succeeding instructions. Conditional branches are executed speculatively in a predicted direction. Instructions are scheduled for execution using dataflow techniques—being executed only when all their needed operands have been computed. Executing a speculative branch that was incorrectly predicted invokes processor state repair. The results of an instruction's execution are held with the instruction in the DCAF, and then retired to permanent storage in program issue order; this provides for automatic register renaming, and for a coherent ("sane") processor state in the case of interrupts or traps.

The Lightning implementation of the Metaflow architecture has been extensively simulated. These studies have produced the following results:

- Lightning can sustain a rate of more than two SPARC instructions executed per clock. The ability to issue and execute instructions out-of-order and speculatively is the key reason for this performance.
- The Lightning processor constantly executes "speculatively".
- Performance is relatively insensitive to memory latency for data in programs having sufficient instruction-level parallelism.
- The limits on performance of the Lightning implementation running the SPECmark benchmark suite were found to be (in decreasing order of importance): memory bandwidth, branch prediction rate, calculated branch address latency, and instruction cache hit rate.

5 Acknowledgements

Val Popescu, Merle Schultz, John Spracklen, Gary Gibson, and Bruce Lightner are co-inventors of the Metaflow architecture. David Isaman is the major contributor to the architecture and design of the Lightning IIU chip. The authors also wish to acknowledge the contributions to the Lightning project made by David Poole of LSI Logic.

References

[1] Val Popescu, Merle Schultz, John Spracklen, Gary Gibson, Bruce Lightner and David Isaman. *The Metaflow Architecture.* Paper submitted to IEEE Micro Magazine (Hot Chips Symposium II Special Edition). Scheduled for June 1991.

[2] *The SPARC Architecture Manual, Version 8.* Published by Sun Microsystems, Inc., Mountain View, CA. Part No: 800-1399-11. June 1990.